

Jake Takahashi, Aaron Castiglione, Jesus Chairez Hernandez, Isaac Jee
Dr. Carreira
CSE 176
17 December 2025

Part 2 Report

There are two files that we need to create multiclassification on. Those are gonna be the MNIST-LeNet5.mat file and the MNIST.mat file. The first one that we are gonna do is the MNIST.mat file.

Our binary classification for the MNIST.mat file is going to be in the `xgb_mnist_pixels.py` file. To start, we first loaded the data into respective training dictionaries and test dictionaries. We then split the training data further into training and validation data sets using a predefined function in the `utils_mnist.py` module.

After this, is the actual part where we clean our data. The first step is to normalize our data using `StandardScaler` from the preprocessing submodule of `scikit-learn`. The next step is interesting. In the first draft of our script, we went immediately to training and hyperparameter tuning after standardizing our data. However, we found that the script had runtimes of up to 2 hours. This immediately gave us the suspicion that we needed to do feature selection. In this case, we can use PCA or LDA, but we decided on PCA. For the PCA, we decided that we want it to keep 80% of the original variance of the data. Originally, we tried with 95% maintenance of original variance, but this still resulted in extremely long run times of up to an hour. 80% maintenance of the original variance gave us runtimes of about 10 minutes, and still had very good performance.

The next step was the actual training of our model. The first step is to define the parameters for our ensemble method; in this case, it is gradient boosting. There are a lot of parameters that we defined, so I will run through them quickly; they are the learning rate, objective function, number of classes from our data, evaluation metric, and the amount of CPU cores we dedicate to training.

Next, we define a dictionary of hyperparameters and the range of those hyper parameters that we I want to search for it. This is because in our next step, we define a model called `RandomizedSearchCV`. This takes my defined ensemble method as input, and the dictionary of hyperparameters that we defined earlier, and trains itself on our training data 4 times, each time using a random combination of the dictionary of hyper parameters that we defined earlier. We finally take the "best" parameters found by our `RandomizedSearchCV` and evaluate it from the training and validation set, and the accuracy in a list to plot on a graph later. We then finally plot it on a matplotlib graph and store it in a .pdf file. The final step is to evaluate the final, final model's performance on our training data. After all of this, we save the results of the best found model in a .txt file and our model and data relevant to properly scaling our model in .pkl files.

Next, we move onto processing the LeNet5 features. The core idea behind this data is that it has already been processed by CNN. We load it in a manner similar to the raw data. Although this data has already been processed by a CNN, we still decide to do feature selection using PCA to save ourselves runtime. A quick series of print statements tell us that this dataset has 60000 training instances, and each instance has 000 features. This is going to be very expensive in terms of computation. Originally, we tried running our model with the data as is, but we had run times in the degree of 2 hours. So, just like the pixel data, we decided to do feature reduction using PCA.

We still decide to do a very aggressive PCA reduction of 80% of the original variance due to our findings in the past that an 80% of the original variance still gives very good performance in terms of model accuracy. PCA,

with the constraint that it had to keep 80% of the original variance, reduced the amount of features of each image from 800 to only 38. This seems like a very aggressive feature reduction, but we will see later that it has very good model accuracy. Just like our raw data classification problem, our next step is to define the parameters of the overall ensemble method.

Our next step was to, again, define a dictionary of different hyper parameters and a range of values for each hyper parameter. We define "n_estimators", which is going to be the number of base learner trees that we will use for our ensemble method. We define "max_depth", which is the maximum depth that we want each base learner tree to grow too. We also define a hyper parameter called colsample_bytree and a range of values for it. This hyper parameter essentially says, of all the features that we have reduced to using PCA, what percentage of those features will be used to train this specific base learner tree? A key thing to note is that if we define colsample_bytree to be 0.80, for example, this means that each base learner tree will train itself on a random 80% of the PCA-reduced features. Again, the percentage of the original data that each base learner tree will train itself on is random at each base learner tree.

Like for the raw data, we define a RandomizedSearchCV model that will allow us to search different combinations of our hyper parameter dictionary to find (not an optimal), but a pretty good estimation of what the best hyper parameters will be. We define it to test 6 random combinations of hyper parameters, and each random combination of hyper parameters is going to be tested using 3 fold cross validation. We also define the amount of cores, this time, to be 2. This is because in our past problem, running the script with all the cores worked fine, but it caused significant lag spikes on our computers. An important parameter to set is random_state.

After a run time of 5 minutes, our randomized search was able to converge on what it believed were the best parameters of its search. It found that a sub_sample of 0.8, 100 base learner trees, a max depth of 8, and a colsample_bytree of 0.6. Our next step was to finalize the hyper parameter tuning process by testing our gradient boosting model with the best found parameters against a different number of base learner trees. For various amounts of base learner trees, we train a gradient boosting model with similarly defined parameters to our earlier defined overall ensemble methods on the training set. For each iteration, after initial training, we then test its performance on the validation set, and then store it in a list. For plotting purposes solely, we then get the model's performance on the test data set.

We then take our stored data, and plot it on a graph using matplotlib. We can see that for a low initial amount of base learner trees, the error rate is relatively high. However, as the amount of base learner trees increases, the validation and test error rate decreases. Also, based on the tree hyper parameter tuning step, the optimal number of base learner trees is 200, based on a validation error rate of 0.0172.

Our final step is to train the final model, so that we can evaluate it on the test set. A key thing to note is that up until this point, we have not used our test set for any training at all. The final step involves combining our training data and validation data into one full data set. We then train a new gradient boosting model on the best hyper parameters found up to this point, and evaluate it on the test set. It is this figure that is important; we get a test error rate of 1.5%. This is good, because it means our model was able to generalize to new data well. It also means that our aggressive PCA reduction of the data was still able to provide very good model accuracy in regards to new data.

After that step, we have a series of print statements that summarizes the run of our script. We also used a python module that started earlier in the script, finding out we had a runtime of around 8-10 minutes.

Sources:

<https://xgboost.readthedocs.io/en/stable/tutorials/model.html>

<https://www.geeksforgeeks.org/machine-learning/xgboost/>

<https://medium.com/data-science/pca-102-should-you-use-pca-how-many-components-to-use-how-to-interpret-them-da0c8e3b11f0>

<https://www.geeksforgeeks.org/machine-learning/stratified-k-fold-cross-validation/>

<https://www.geeksforgeeks.org/machine-learning/cross-validation-machine-learning/>

Confusion Matrices:

Pixels Confusion Matrix

```
Confusion Matrix:
[[1118   4   1   1   1   4   0   6   0   0]
 [   1  971  17   6   1   4   8  18   0   6]
 [   0  10  954   1  14   2  12  13   4   0]
 [   0   4   1  935   0  11   5   3  22   1]
 [   0   1  25   5  824  10   2  13   6   6]
 [   2   2   0   4  11  929   0   4   0   6]
 [   8  16   6   7   1   0  962   4  24   0]
 [   0   6  18   6  12   3   8  910   6   5]
 [   5   3  10  25   5   1  10   7  938   5]
 [   0   1   1   1   3   5   1   2   1  965]]
```

Lenet Confusion Matrix:

```
=== Confusion Matrix ===
Rows: True labels, Columns: Predicted labels
  0   1   2   3   4   5   6   7   8   9
0: 1129   1   2   0   0   1   0   2   0   0
1:   0 1020   0   0   0   1   8   2   0   1
2:   0   3  996   0   3   0   5   2   0   1
3:   0   2   0  968   0   2   0   1   9   0
4:   0   0  10   0  876   2   1   0   1   2
5:   3   0   0   1   2  945   0   1   0   6
6:   2  10   2   1   1   0 1005   2   4   1
7:   0   2   2   1   2   0   3  960   1   3
8:   5   1   4   9   0   2   9   2  975   2
9:   0   1   0   0   0   0   1   3   0  975
```